DEVOPS-003 — Linux Networking & Shell Scripting Fundamentals

Task 4 — Install and Configure OpenSSH (1 Hour)

This log documents installing the OpenSSH server, enabling and starting the sshd service, and verifying it by logging into the machine over SSH as a non-root user, running a command, and logging back out.

Environment Note

This was performed in a containerized Linux (Ubuntu 24.04) sandbox that runs without systemd as PID 1 (it uses a lightweight container init instead). Because of that, systemctl status/enable/start ssh fail with 'System has not been booted with systemd as init system' — this is a property of running Linux inside a minimal container, not an error in the steps. The service was instead started with the equivalent legacy SysV wrapper, service ssh start, which controls the same sshd daemon. On a normal Ubuntu VM or physical machine (which boots with systemd as PID 1), the exact systemctl commands from the task sheet work as written — the underlying install, config, and verification steps are identical either way.

1. Install SSH Server

Command	**`sudo apt update`**
Command	**`sudo apt install openssh-server`**

```
$ apt-get install -y openssh-server
Setting up openssh-sftp-server (1:9.6p1-3ubuntu13.18) ...
Setting up openssh-server (1:9.6p1-3ubuntu13.18) ...
Creating SSH2 RSA key; this may take some time ...
3072 SHA256:S0MiZPPb3GWf7c9ifYoBn3IhhPTfPqkEg2IZe8CmX7A root@vm (RSA)
Creating SSH2 ECDSA key; this may take some time ...
256 SHA256:64zYSeW9/14OvUH21gA+k2sufm57i/OPHcy8ddWktRc root@vm (ECDSA)
Creating SSH2 ED25519 key; this may take some time ...
256 SHA256:zJMFLZsffgW3Ke7sIuMpFX9BhV/aKIBcD3j9z3sSGEA root@vm (ED25519)
Created symlink /etc/systemd/system/sockets.target.wants/ssh.socket -> ...
```

The install pulls in openssh-server and openssh-sftp-server, and generates three host key pairs (RSA, ECDSA, ED25519) that uniquely identify this machine to SSH clients — these are what a client's known_hosts fingerprint check is comparing against.

2. Verify / Enable / Start

Command	**`sudo systemctl enable ssh`** *Registers sshd to auto-start on boot.*
Command	**`sudo systemctl start ssh`** *Starts the service now (used 'service ssh start' here — see Environment Note above).*
Command	**`systemctl status ssh`** *Confirms the daemon is active and shows its recent log lines.*

```
$ systemctl enable ssh
Created symlink /etc/systemd/system/sshd.service -> /usr/lib/systemd/system/ssh.service.
Created symlink /etc/systemd/system/multi-user.target.wants/ssh.service -> ...

$ service ssh start
 * Starting OpenBSD Secure Shell server sshd
   ...done.

$ service ssh status
```

```
 * sshd is running

$ ss -tlnp | grep :22
LISTEN  0  128  0.0.0.0:22  0.0.0.0:*  users:(("sshd",pid=905,fd=3))

$ sshd -V
OpenSSH_9.6p1 Ubuntu-3ubuntu13.18, OpenSSL 3.0.13 30 Jan 2024
```

ss -tlnp confirms sshd is bound to and LISTENing on TCP port 22 across all interfaces (0.0.0.0), which is the actual proof the service is reachable — 'sshd is running' alone only confirms the process exists, not that it's accepting connections.

3. Screenshot — SSH Login

Connecting as a non-root user (devuser) to the local machine with ssh username@localhost, authenticating with a password, then running a command to prove the session is live.

```
$ ssh devuser@localhost
devuser@localhost's password:
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.18.5-fc-v20 x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

devuser@vm:~$ whoami
devuser
devuser@vm:~$ echo Connected via SSH to $(hostname)
Connected via SSH to vm
```

4. Screenshot — SSH Logout

Ending the session cleanly from inside the remote shell with the logout command.

```
devuser@vm:~$ logout
Connection to localhost closed.
```

5. Screenshot — SSH Service Status

Re-checking the daemon's status after the session ended, to confirm sshd itself keeps running and continues listening for the next connection (it doesn't stop just because one client disconnected).

```
$ service ssh status
 * sshd is running

$ ss -tlnp | grep :22
LISTEN  0  128  0.0.0.0:22  0.0.0.0:*  users:(("sshd",pid=905,fd=3))
```

Connect using

ssh devuser@localhost — connects as the local user 'devuser' over SSH to this same machine (127.0.0.1), authenticating with the account password set for that user. The task sheet also notes: 'If another Linux VM is available, connect to that machine' — the exact same command works against a remote host, just swap localhost for that machine's hostname or IP (e.g. ssh devuser@192.168.1.20), provided sshd is installed and running there too and the network/firewall allows port 22.

Summary

openssh-server was installed, its service (sshd) was enabled and started, and a full round trip — login as a non-root user, run a command, logout — was verified end-to-end. ss -tlnp additionally confirmed sshd is actually listening on TCP/22, and the service remained running after the client disconnected, showing the daemon serves multiple sessions rather than exiting after one.